

第零章：实验环境搭建

实验环境配置

本节我们将完成环境配置并成功运行 uCore-Tutorial-2025S。整个流程分为下面几个部分：

- 系统环境配置
- Riscv下 C 开发环境配置
- Qemu 模拟器安装
- 其他工具安装
- 运行 uCore-Tutorial-2025S

目前实验仅支持 Ubuntu 24.04 操作系统。对于 Windows10 和 macOS 上的用户，可以使用 VMware 或 VirtualBox 安装一台 Ubuntu 24.04 虚拟机并在上面进行实验。

系统环境配置

Windows10 的用户可以通过系统内置的 WSL2 虚拟机（请不要使用 WSL1）来安装 Ubuntu 24.04。步骤如下：

1. **升级 Windows 10 到最新版**（Windows 10 版本 18917 或以后的内部版本）。注意，如果不是 Windows 10 专业版，可能需要手动更新，在微软官网下载。升级之后，可以在 PowerShell 中输入 `winver` 命令来查看内部版本号。
2. **加入开发者模式**：「Windows 设置 > 更新和安全 > Windows 预览体验计划」处选择加入“Dev 开发者模式”。
3. **以管理员身份打开 PowerShell 终端并输入以下命令**：

```
# 启用 windows 功能：“适用于 Linux 的 windows 子系统”
>> dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

# 启用 windows 功能：“已安装的虚拟机平台”
>> dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart

# <Distro> 改为对应从微软应用商店安装的 Linux 版本名，比如：`wsl --set-version Ubuntu 2`
# 如果你没有提前从微软应用商店安装任何 Linux 版本，请跳过此步骤
>> wsl --set-version <Distro> 2

# 设置默认为 WSL 2，如果 windows 版本不够，这条命令会出错
>> wsl --set-default-version 2
```

4. **下载 Linux 内核安装包**
5. **在微软商店（Microsoft Store）中搜索并安装 Ubuntu18.04 / 20.04。**

C 开发环境配置

这一大步和下面的 Docker 安装方式大家可以选用一种。

首先，我们需要安装好 RISC-V 配套的 gcc。首先选择一个位置放置 gcc 的可执行二进制文件。我们选择一个常用的位置。这个位置也可以大家自己指定。

```
cd /usr/local
```

之后直接下载预编译好的Risc-v工具链：

```
sudo wget https://static.dev.sifive.com/dev-tools/freedom-tools/v2020.08/riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz
```

解压缩：

```
sudo tar xzf riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz
```

文件名改短：

```
sudo mv riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14 riscv64-unknown-elf-gcc
```

这里就算安装完成了。接下来我们要把gcc的二进制文件路径添加到PATH之中，这样我们才能在任意目录直接运行它。将以下指令添加到home下的.bashrc之中可以一劳永逸地添加(如果你使用的是自己的路径请更换路径的前缀/usr/local到你自己的路径)：

```
export PATH="/usr/local/riscv64-unknown-elf-gcc/bin:$PATH"
```

接下来，继续安装用于交叉编译的musl-gcc，这里我们仍然使用/usr/local存放它。下面的步骤和上一步的安装是一样的：

```
cd /usr/local
# 如果链接失效，可以用官网链接下载并提醒助教更新网盘地址： https://more.musl.cc/10/x86_64-linux-musl/riscv64-linux-musl-cross.tgz
# 注意这里是 10.2.1 版本，如果从其他地方下载请务必注意版本号一致
sudo wget -O riscv64-linux-musl-cross.tgz
"https://cloud.tsinghua.edu.cn/f/fb3c598e7e214a828e6b/?dl=1"

sudo tar xzf riscv64-linux-musl-cross.tgz
```

将路径添加到PATH之中：

```
export PATH="/usr/local/riscv64-linux-musl-cross/bin:$PATH"
```

我们的项目使用cmake搭建，因此还需要安装cmake。

```
sudo apt install cmake
```

如果是第一次启动虚拟机，需要执行：

```
sudo apt update
sudo apt upgrade
```

至于 C 开发环境，推荐直接使用Vscode作为IDE来进行。可以在其中安装C的插件来使用自动补全和lint。

Docker 环境安装（可选，已完成上述步骤的可以忽略）

使用配置好的Docker容器可以免于自己安装上面的一系列包，当然会多出配置Docker的工作量。

1. 首先安装Docker: <https://www.docker.com/get-started>

2. 接着拉取我们的Docker镜像文件:

```
docker pull tclab/ucore-env
```

3. 查看下载的镜像文件: `docker image ls tclab/ucore-env`

4. 使用下载的镜像文件创建一个名字叫container_name（可自己指定）的容器，并获得容器的bash shell:

```
docker run -it --name container_name image_name /bin/bash
```

一些常用的指令:

检查运行中的容器

```
docker ps
```

检查所有容器（包括停止了）

```
docker ps -a
```

停止 / 启动容器

```
docker stop/start container_name
```

获取一个运行中的docker容器的bash shell:

```
docker exec -it container_name /bin/bash
```

删除一个已经停止的容器:

```
docker rm container_name
```

5. 之后就是把我们的宿主机的文件挂载在docker容器上。在使用 `docker run` 启动容器时，你可以将目录挂载到容器上，这样就可以从docker容器访问到本地主机的某个文件夹。

```
docker run -it --name container_name --mount type=bind,src=[absolute path of folder in host machine],dst=[absolute path in container] image_name /bin/bash
```

Qemu 模拟器安装

我们推荐使用 Qemu 7.0.0 版本及以上进行实验，而很多 Linux 发行版的软件包管理器默认软件源中的 Qemu 版本过低，可能无法顺利实验。因此我们需要从源码手动编译安装 Qemu 模拟器。

首先我们安装依赖包，获取 Qemu 源代码并手动编译:

安装编译所需的依赖包

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev libmpfr-dev libgmp-dev \
gawk build-essential bison flex texinfo gperf libtool patchutils bc ninja-build \
```

```

zlib1g-dev libexpat-dev pkg-config libglib2.0-dev libpixman-1-dev git tmux
python3

# 下载源码包
# 如果链接失效, 可以使用官网链接下载并提醒助教更新云盘地址: https://download.qemu.org/qemu-7.0.0.tar.xz
wget -O qemu-7.0.0.tar.xz "https://cloud.tsinghua.edu.cn/f/8ba524dbede24ce79d06/?dl=1"

# 解压
tar xJf qemu-7.0.0.tar.xz

# 编译安装并配置 RISC-V 支持
cd qemu-7.0.0
./configure --target-list=riscv64-sofmmu,riscv64-linux-user
make -j$(nproc)

```

注解

注意, 上面的依赖包可能并不完全, 比如在 Ubuntu 24.04 上:

- 出现 `ERROR: pkg-config binary 'pkg-config' not found` 时, 可以安装 `pkg-> config` 包;
- 出现 `ERROR: glib-2.48 gthread-2.0 is required to compile QEMU` 时, 可以安装 `libglib2.0-dev` 包;
- 出现 `ERROR: pixman >= 0.21.8 not present` 时, 可以安装 `libpixman-1-dev` 包。

另外一些 Linux 发行版编译 Qemu 的依赖包可以从 [这里](#) 找到。

GCC 11 可能无法正常编译 Qemu7, 而 GCC 9.3.0 (Ubuntu 24.04 自带) 及 GCC 10.3.0 经测试可以编译, 请自行选择合适的编译器版本。

之后我们可以在同目录下 `sudo make install` 将 Qemu 安装到 `/usr/local/bin` 目录下, 但这样经常会引起冲突。个人来说更习惯的做法是, 编辑 `~/.bashrc` 文件 (如果使用的是默认的 bash 终端), 在文件的末尾加入几行:

```

# 注意 $HOME 是 Linux 自动设置的表示你家目录的环境变量, 你也可以根据实际位置灵活调整
export PATH="$HOME/os-env/qemu-7.0.0/build/:$PATH"
export PATH="$HOME/os-env/qemu-7.0.0/build/riscv64-sofmmu:$PATH"
export PATH="$HOME/os-env/qemu-7.0.0/build/riscv64-linux-user:$PATH"

```

随后即可在当前终端 `source ~/.bashrc` 更新系统路径, 或者直接重启一个新的终端。

此时我们可以确认 Qemu 的版本:

```

qemu-system-riscv64 --version
qemu-riscv64 --version

```

注: 有同学在 Ubuntu 24.04 上遇到了执行 `qemu-system-riscv64 --version` 报错找不到 `libtinfow.so.6`, 可以通过如下软连接修复:

```

sudo ln -s /lib/x86_64-linux-gnu/libtic.so.6.2 /lib/x86_64-linux-gnu/libtinfow.so.6

```

GDB 调试支持

在 `os` 目录下 `make debug` 可以调试我们的内核，这里需要基于 riscv64 平台的 gdb 调试器 `riscv64-unknown-elf-gdb`。该调试器包含在 riscv64 gcc 工具链中，工具链的预编译版本可以在如下链接处下载：

- Ubuntu 平台
- macOS 平台
- Windows 平台
- CentOS 平台

解压后在 `bin` 目录下即可找到 `riscv64-unknown-elf-gdb` 以及另外一些常用工具 `objcopy/objdump/readelf` 等。

VSCode 可视化调试支持

本节将介绍如何在VSCode可视化环境中进行调试。可以使用使用网页版的Codespace，也可以使用Codespace + 本地VScode（具体方法就是在打开Codespace时，点击Open In Visual Studio Code即可）。

要在Codespace中结合vscode使用GDB进行调试，需要确保Codespace中有GDB、QEMU等相关环境。如果没有环境可以在master分支下执行环境配置指令：

```
git checkout master // 切换到master分支
make codespaces_setenv // 配置Codespace环境
```

然后设置“add-auto-load-safe-path”路径，将“add-auto-load-safe-path /workspaces/YOUR_REPO_NAME/.gdbinit”语句添加到 `/home/codespace/.gdbinit` 文件中，如果该 `/root/codespace` 路径下没有.gdbinit文件，则创建后添加前面的语句。语句中“YOUR_REPO_NAME”则是你对应仓库的名字。

最后启动GDB调试：

```
make debug
```

之后则可以设置断点进行GDB调试。

在 Qemu 平台上运行 uCore-Tutorial-2023S

到这里，恭喜你完成了实验环境的配置，可以开始阅读教程的正文部分了！可以直接clone下面的仓库来开始OS之旅：

```
git clone https://github.com/LearningOS/uCore-Tutorial-Code-2023S.git
cd uCore-Tutorial-Code-2023S
```

其他的章节需要处理用户代码，我们可以先运行不需要处理用户代码的 `ch1` 分支：

```
git checkout ch1
make run LOG=debug
```

忽略掉编译输出后，你应该得到如下的输出，这表示 uCore 已经成功运行。

```
[rustsbi] RustSBI version 0.1.1
```

	-	\						/			/			-	\	
	_)								(----	----		----	----		_)	
		/						\	\				\	\		<
	\	\-----		'--'		-----)					.-----)			_)		
	_ `	,_____		_____/		_____/		_		_____/			_____/	_		

```
[rustsbi] Platform: QEMU (Version 0.1.0)
```

```
[rustsbi] misa: RV64ACDFIMSU
```

```
[rustsbi] midleg: 0x222
```

```
[rustsbi] medeleg: 0xb1ab
```

```
[rustsbi-dtb] Hart count: cluster0 with 1 cores
```

```
[rustsbi] Kernel entry: 0x80200000
```

hello world!

```
[ERROR 0]stext: 0x0000000080200000
```

```
[WARN 0]etext: 0x0000000080201000
```

```
[INFO 0]sroda: 0x0000000080201000
```

```
[DEBUG 0]eroda: 0x0000000080202000
```

```
[DEBUG 0]sdata: 0x0000000080202000
```

```
[INFO 0]edata: 0x0000000080202000
```

```
[WARN 0]sbss : 0x0000000080212000
```

```
[ERROR 0]ebss : 0x0000000080212000
```

```
[PANIC 0] os/main.c:39: ALL DONE
```

注解

1. 上方的输出是比较早期版本的，我们现在使用的 rustsbi-qemu 已经更新至最新版（2022-10），与之有些微差异之处。
2. **退出 qemu 的方法**：如果是正常推出，uCore 会自动关闭 qemu，但如果 os 跑飞了，我们不能通过 Ctrl + C 来推出。此时可以先按下 Ctrl+A，再按下 X 来退出 Qemu。